

Week 1 Revision Guide

Data Foundations · Days 01 – 07

Concepts · Code · Real World Problems · Interview Prep

WHAT'S INSIDE

DAY 01

**What is Data
in ML**

DAY 02

**Data Types &
Distributions**

DAY 03

**Descriptive
Statistics**

DAY 04

**Exploratory
Data Analysis**

DAY 05

Missing Data

DAY 06

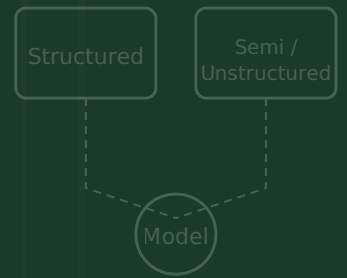
**Outlier
Detection**

DAY 07

Data Cleaning

BONUS

**MNC Interview
Q&A**



Week 1 · Data Foundations

Types of Data

THE BIG 3 — EVERY MLE MUST KNOW THESE

Structured

Rows & columns. Fixed schema. Every field labeled.

Examples: customer transactions, sensor readings, stock prices, CSV files

→ loads directly into pandas

Semi-Structured

Loose schema. Fields can vary per record. Has some organisation but no strict rules.

Examples: JSON logs, XML files, emails, resumes

→ parse & flatten first

Unstructured

No fixed format at all. Model cannot consume it directly — needs feature extraction first.

Examples: images, audio, raw text, video

→ extract features first

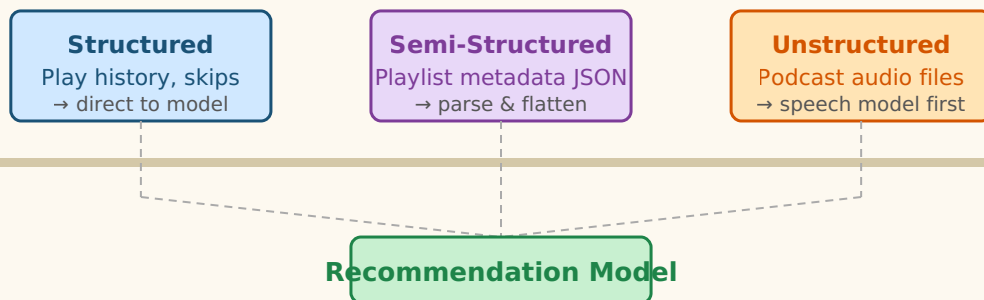
THE ANALOGY

Structured = Government form — every field has a label and a box to fill

Semi-structured = Resume — everyone follows a rough format, no two look identical

Unstructured = Handwritten letter — no fixed format, must interpret from scratch

REAL WORLD — HOW SPOTIFY HANDLES ALL 3 SIMULTANEOUSLY



3 separate ingestion pipelines. The architecture is the product — not the algorithm.

KEY SYNTAX

```
import pandas as pd import json from PIL import Image
# Structured
df = pd.read_csv("data.csv") # check df.shape and df.dtypes
# Semi-structured
meta = json.load(open("meta.json")) # print(meta.keys())
# Unstructured
img = Image.open("cover.jpg") # print(img.size, img.mode)
```

Golden Rule

Unstructured data **always needs a feature extraction step** before any model. Images → pixel tensors. Text → embeddings. Audio → spectrograms. Without this step, your model has nothing to learn from.

DAY 01 · Q1

GOOGLE — ML ENGINEER ROUND

You receive a dataset with structured, unstructured, and semi-structured data. How do you approach building a pipeline for it?

ANSWER DIRECTION

Identify each data type and source system. Build separate ingestion pipelines per type with schema validation at each entry point. For unstructured data, add a feature extraction step (text → embeddings, images → tensors) before model input. Join processed feature vectors at the end. Mention update frequencies — structured data may update daily, chat logs in real time. Mismatched update frequencies break production pipelines.

DAY 01 · Q2

MICROSOFT — SYSTEM DESIGN ROUND

Why can't you feed raw image data directly into a logistic regression model? What needs to happen first?

ANSWER DIRECTION

Logistic regression requires a fixed-length numerical feature vector as input. A raw image is an unstructured grid of pixels with no inherent "feature" meaning the model can use. You need a feature extraction step: flatten the image into a pixel array, apply a pretrained CNN to extract embeddings, or use HOG/SIFT for traditional feature extraction. Only after this representation step does the data become consumable by a classical ML model.

Data Types

THE 4 SEMANTIC TYPES

Numerical

Continuous: height, salary, temperature — infinite values between two points

Discrete: # rooms, order count — countable

Arithmetic is meaningful ✓

Categorical (Nominal)

No inherent order at all. Just labels.

City names, colours, product category, job title

Just labels — no math ✓

Ordinal

Has a natural order but gaps between values are NOT necessarily equal.

Star ratings (1-5), Education level, Pain scale

Order matters, gaps don't ✓

Boolean

Exactly two states: True / False

is_fraud, is_active, email_opened, clicked

Binary signal only ✓

⚠️ CLASSIC PRODUCTION BUG

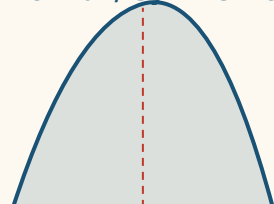
Storing zip codes as **integers**: your model learns that zip 400001 is "greater than" zip 110001.

This is mathematically nonsense but the model will happily use it. Storing star ratings as

integers: the model assumes rating 4 is exactly twice as good as rating 2. It isn't. One dtype fix, caught in EDA, can prevent weeks of debugging.

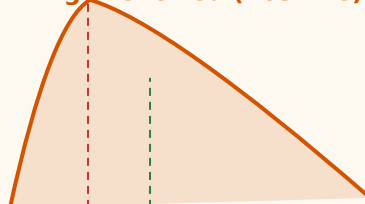
DISTRIBUTIONS — WHAT SHAPE IS YOUR DATA?

Normal / Symmetric



mean $\approx$ median $\approx$ mode

Right-Skewed (Positive)



mode mean
mean > median — long right tail

House prices!



→ log transform fixes

KEY SYNTAX

```
# ALWAYS audit dtypes before any preprocessing
print df.dtypes
df['zip'] = df['zip'].astype('str')
df['rating'] = df['rating'].astype('category')
# Plot + check skew (|skew| > 1 = flag it!)
df['price'].hist(bins=30)
print df['price'].skew
```

Real World — Airbnb

Airbnb treats "number of bedrooms" as numerical but "neighbourhood" as categorical. A type mismatch in just one feature column degraded their price suggestion model by 11% before it was caught in monitoring. Documented in their engineering blog.

INTERVIEW QUESTIONS

DAY 02 · Q1

FLIPKART — DATA SCIENTIST INTERVIEW

You have a "star rating" column (1-5). Your colleague encodes it as a numerical feature. What's wrong and how do you fix it?

ANSWER DIRECTION

Star rating is ordinal — the gap between 1→2 stars may not equal the gap between 4→5 stars. For linear models: one-hot encode to remove false numerical distance assumptions. For tree-based models (Random Forest, XGBoost): ordinal encoding is fine since trees split on thresholds, not numeric gaps. Always choose encoding strategy based on model type.

DAY 02 · Q2

AMAZON — ML ENGINEER ROUND

Why would the mean and median of a salary column be very different, and what does that tell you about the data?

ANSWER DIRECTION

High positive skewness. Mean is sensitive to extreme values (a few very high earners pull it upward), while median is robust. Mean >> median means the distribution has a long right tail. Actionable: use median as the central tendency measure for reporting, use median for imputation, and consider log-transforming the column before feeding into a linear model.

The 6 Stats You Run First

CORE MEASURES — BEFORE TOUCHING ANY MODEL

Mean (μ)

Sum of all values ÷ count

⚠ Sensitive to outliers

```
df['col'].mean()
```

Median

Middle value after sorting

✓ Robust to outliers

```
df['col'].median()
```

Std Dev (σ)

Spread around the mean
= $\sqrt{\text{Variance}}$

✓ Spread in original units

```
df['col'].std()
```

Variance (σ^2)

Avg squared deviation from mean

⚠ Squared units (harder to interpret)

```
df['col'].var()
```

Skewness

Asymmetry of distribution

$|\text{skew}| > 1 \rightarrow$ flag it!

+ve = right tail

-ve = left tail

```
df['col'].skew()
```

Kurtosis

Heaviness of tails

High kurtosis = outlier risk

Normal = 3 (excess = 0)

```
df['col'].kurtosis()
```

df.describe() → gives mean, std, min, 25%, 50%, 75%, max in one call

LOG TRANSFORM — FIX SKEWED DATA

KAGGLE HOUSE PRICES DATASET

SalePrice column has skewness = 1.88 (heavily right-skewed). Applying `np.log1p()` brings skewness down to 0.12. That single transformation improved RMSE by 18% without changing the model at all. This is documented by multiple Kaggle grandmasters on this exact dataset.

KEY SYNTAX

```
# Get all stats at once
print df.describe()

# Check skewness per column
skewed_cols = df.skew().abs() > 1

# Fix with log transform
import numpy as np
df['SalePrice_log'] = np.log1p(df['SalePrice']) # log1p handles zeros safely
```

Why use log1p not log?

$\log(0)$ = undefined (breaks on zero values).
 $\log1p(x) = \log(1+x)$, safely handles zeros and near-zeros. Use it on any column that might contain 0.

Mean vs Median choice:

Amazon uses *median* delivery time in SLA reports, not mean. One extreme delay inflates mean. Median stays stable. Same principle applies to imputation decisions.

INTERVIEW QUESTIONS

DAY 03 · Q1

AMAZON — DATA SCIENCE INTERVIEW

Your target variable has a skewness of 2.3. What do you do before training a linear regression model?

ANSWER DIRECTION

Apply log or Box-Cox transformation to the target variable. Train the model on the transformed values. When reporting predictions, inverse-transform them (`np.exp(m1.predict())` if `log1p` was used). Measure skewness before and after to confirm the fix worked. Reason: linear regression assumes normally distributed residuals — high target skewness usually causes skewed residuals, violating this assumption.

DAY 03 · Q2

PAYTM — ML ENGINEER INTERVIEW

Two features in your dataset are 95% correlated. What do you do and why?

ANSWER DIRECTION

High multicollinearity. In linear models, it inflates coefficient variance and makes coefficients unreliable. Fix: drop one of the two features, or apply PCA to combine them into one component. In tree-based models (Random Forest, XGBoost) it matters less — trees are not affected by multicollinearity — but correlated features split the feature importance score, making interpretation harder. Always state which model you're using before choosing the fix.

EDA — The Doctor's Checkup

THE ANALOGY

Skipping EDA and jumping straight to modelling is like a doctor prescribing medication without running any tests first. You might get lucky. You probably won't. And when the model fails, you won't know why.

THE EDA WORKFLOW — ALWAYS IN THIS ORDER



EDA generates hypotheses. The model tests them.

THE ACCURACY PARADOX — WHY EDA SAVES LIVES

PRODUCTION FAILURE MODE

A churn model reports 94% accuracy. Team ships it. Model predicts "No Churn" for every single customer. Accuracy is 94% because 94% of customers don't churn anyway. The model learned nothing. **One line of EDA** — `df['churn'].value_counts()` — would have caught the class imbalance before a single training epoch ran.

KEY SYNTAX

```
import seaborn as sns
# Correlation heatmap
sns heatmap df corr      annot=True  cmap='coolwarm'
# Class balance check — ALWAYS run this first on classification!
print df 'target'  value_counts normalize=True
# Auto-EDA in 3 lines
from ydata_profiling import ProfileReport
ProfileReport df  to_file "eda_report.html"
```

Instacart Kaggle Competition Finding

Top 10 teams all spent 30-40% of total project time on EDA before writing a single model. The winning feature — "time since last order" — was discovered entirely through EDA, not model iteration. The pattern was hiding in the data, visible only to whoever looked carefully before training.

INTERVIEW QUESTIONS

DAY 04 · Q1

MICROSOFT — ML ENGINEER INTERVIEW

Walk me through how you approach EDA on a completely new dataset you've never seen before.

ANSWER DIRECTION

Step 1: shape, dtypes, missing value counts (df.info()). Step 2: univariate — histograms for numerical, value_counts for categorical. Step 3: target variable — distribution and class balance check. Step 4: bivariate — correlation heatmap, scatter plots for top correlated feature pairs. Step 5: pairplot on top features vs target. Flag every unexpected pattern before touching a model. EDA should end with a list of hypotheses to test, not just plots.

DAY 04 · Q2

RAZORPAY — DATA SCIENTIST ROUND

Your fraud detection model has 99.5% accuracy but the business says it's useless. Why?

ANSWER DIRECTION

Classic accuracy paradox with class imbalance. If only 0.5% of transactions are fraud, predicting "no fraud" for everything gives 99.5% accuracy while catching zero fraud cases. Accuracy is the wrong metric here. Should use Precision, Recall, F1, or AUC-ROC. EDA on the target column (value_counts) before model training would have immediately flagged this imbalance and forced a better evaluation strategy.

Missing Data — 3 Types, 3 Strategies

THE 3 TYPES — IDENTIFYING WHICH ONE CHANGES EVERYTHING

MCAR

Missing Completely At Random

No relationship to any other variable. Completely random.

Analogy: forms randomly lost in the post

Safe: drop rows or simple mean/median impute

MAR

Missing At Random

Related to other observed columns but not to the missing value itself.

Analogy: men less likely to report weight (predictable from gender)

Use: KNN or iterative imputer

MNAR

Missing Not At Random

Missing because of the value itself. The hardest case.

Analogy: sick people don't attend health surveys

Use: indicator column + impute

3 IMPUTATION STRATEGIES

Simple Imputation

Mean / Median / Mode
MCAR only
⚠️ distorts distribution

KNN Imputer

Uses k nearest neighbours
Good for MAR data
✓ preserves correlations

Missingness Indicator

Add binary was_missing col
For MNAR data
✓ model learns from gap

LINKEDIN'S MNAR SOLUTION

LinkedIn's feed ranking team documented that engagement data (clicks, likes) is MNAR — users who don't engage don't produce data. Their fix: add a binary "was_this_observed" column alongside imputed values. The model then learns from both the imputed value AND the fact that it was missing. This is now standard practice in production recommender systems.

KEY SYNTAX

```
# Visualise missingness pattern
import missingno as msno
msno.matrix(df) # white bars = missing values

# KNN Imputation (fit on train only – avoid leakage!)
from sklearn.impute import KNNImputer
imp = KNNImputer(n_neighbors=5)
X_train = imp.fit_transform(X_train) # fit HERE
X_test = imp.transform(X_test) # transform only, no fit!

# Missingness indicator column
df['age_missing'] = df['age'].isnull().astype('int')
```

The Data Leakage Warning

Always fit the imputer on training data only, then transform both train and test separately. Fitting on the combined dataset means test data influenced the imputation — that's leakage. Your model will look better in evaluation than it is in production.

INTERVIEW QUESTIONS

DAY 05 · Q1

GOOGLE — SENIOR MLE ROUND

You have a column with 35% missing values. How do you decide whether to impute or drop it?

ANSWER DIRECTION

First identify missingness type. If MCAR: dropping rows is defensible (no bias introduced). If MAR or MNAR: dropping rows introduces selection bias. Then check feature importance with a quick RandomForest — low importance column can be dropped. High importance: impute using MCAR→mean, MAR→KNN, MNAR→indicator column. In production always add the missingness indicator alongside imputed values so the model can learn from both.

DAY 05 · Q2

SWIGGY — DATA SCIENTIST INTERVIEW

Why is mean imputation dangerous for MNAR data? Give a concrete example.

ANSWER DIRECTION

MNAR means the value is missing because of what it would have been. Imputing the mean assumes missingness is random — it isn't. Example: hospital readmission model. Lab test results missing for 30% of patients. They were discharged quickly (healthier), so test wasn't ordered. Imputing with the column mean assigns average lab values to the healthiest patients, teaching the model that average values = low readmission risk. The bias is systematic and invisible until you look at missingness type. Documented failure pattern in clinical ML literature.

Outliers — Signal or Noise?

THE ANALOGY — FROM OUR OWN STORY

My friends eat biryani, pizza, and fries every Friday. I eat salad and dal. I'm the statistical outlier at the lunch table.

Am I noise to be removed? Or the most interesting data point in a study about peer pressure and food choices?

The context decides. Never the algorithm alone.

3 DETECTION METHODS

IQR Method

Flag if:

$$x < Q1 - 1.5 \times IQR$$

$$x > Q3 + 1.5 \times IQR$$

- ✓ Non-parametric
- ✓ Works on any distribution
- ✓ Default first choice

Z-Score

Flag if:

$$|z| > 3$$

$$z = (x - \mu) / \sigma$$

- ✓ Simple and fast
- ⚠ Assumes normal distribution
- ⚠ Sensitive to extreme values

Isolation Forest

Tree-based. Isolates anomalies via random feature splits. Anomalies are easier to isolate.

- ✓ Multivariate detection
- ✓ No distribution assumptions
- ✓ Scales to large datasets

REMOVE VS KEEP — THE DOMAIN DECISION

Remove / Cap ✕

Data entry errors
Sensor malfunctions
House price prediction (noise)



Keep ✓

Fraud detection (Visa)
Holiday delivery time spikes
Rare disease signals in clinical ML

VISA'S FRAUD SYSTEM

Visa processes 65,000 transactions per second. Their ML system uses Isolation Forest + supervised models to flag outliers in under 1 millisecond. The outlier IS the product — a transaction that deviates from normal behaviour is precisely what the model needs to find. Removing outliers would make the entire fraud detection system useless.

KEY SYNTAX

```
# IQR Method
Q1 = df['col'].quantile(0.25)  Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
mask = df['col'] > Q1 - 1.5*IQR & df['col'] < Q3 + 1.5*IQR

# Isolation Forest
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05, random_state=42)
df['is_outlier'] = iso.fit_predict(df)  # -1 = outlier, 1 = normal
```

The Production Trap — Delivery Model

An MLE builds a delivery time model and removes all outliers with IQR before training. The model looks great on paper. It fails every holiday season because holiday deliveries take 3–4× longer — statistically outliers, but operationally the exact cases the business needs accurate predictions for. Always evaluate model performance on your "outlier" subset separately before deciding to remove.

INTERVIEW QUESTIONS

DAY 06 · Q1

PAYTM — ML ENGINEER INTERVIEW

You detect that 8% of your training data are outliers. What is your process?

ANSWER DIRECTION

Step 1: domain check — are these data errors (sensor faults, entry mistakes) or real extreme events? For errors: remove or correct. For real extreme events (holiday spikes, rare fraud): keep and evaluate model performance on the outlier subset separately. If keeping them hurts overall performance, consider two models: one for normal range, one for extremes. Never remove purely because they're statistically anomalous without domain understanding first.

DAY 06 · Q2

FLIPKART — DATA SCIENTIST ROUND

What's the difference between removing an outlier and capping it? When would you use each?

ANSWER DIRECTION

Removing deletes the entire row — reduces dataset size. Capping (Winsorizing) replaces extreme values with a threshold (e.g. 99th percentile) and keeps the row. Use capping in production pipelines where data volume matters and losing rows is expensive. Use removal only when the row is genuinely erroneous and keeping it (even capped) would introduce noise. Capping also prevents the model from seeing impossible values at inference time without reducing your training data.

Data Cleaning — The 5 Issues

WHY THIS MATTERS MOST

A model trained on dirty data doesn't fail loudly. It gives you **confident wrong answers**. Google's Street View team found that cleaning inconsistent address formatting improved model accuracy from 89% to 96.8% — same architecture, same algorithm. Only the cleaning changed.

THE 5 ISSUES IN EVERY PRODUCTION DATASET

① Exact Duplicates

Inflate training data. A model trained on 5 copies of the same row learns it 5× harder. Skews your loss function silently.

```
df.duplicated().sum()
df.drop_duplicates()
```

② Inconsistent Formatting

"Mumbai", "mumbai", "MUMBAI", " mumbai" → 4 phantom categories. Your model treats them as completely different cities.

```
df['city'].str.strip().str.title()
```

③ Wrong Data Types

ZIP codes as integers. Phone as float. Dates as strings. Silently breaks distance-based models and joins.

```
df['zip'].astype('str')
pd.to_datetime(df['date'])
```

④ Contradictory Records

age=25 but birth_year=1960. Internally inconsistent rows. A model learns a false signal from these.

Cross-check columns. Trust the ground truth one.

```
df['age_from_birth'] = 2024 - df['birth_year']
flag where |age - age_from_birth| > 2
```

⑤ Structural Errors

Hidden whitespace in emails: " user@gmail.com "
Mixed case: "user@GMAIL.COM"
String NAs: "NA", "null", "N/A" not parsed as NaN

```
df['email'].str.strip().str.lower()
df['phone'].replace('NA', np.nan)
```

FROM OUR DATASET — WHAT WE FOUND

City Column (Before)

"Mumbai" · "mumbai"
"MUMBAI" · " mumbai"
"Mumbai " → 6 categories
Model sees 6 cities ✗



City Column (After)

strip() + title() + fuzzy map
"Mumbai" · "Pune"
"Delhi" · "Bangalore"
Model sees 4 cities ✓

Our Dirty Dataset

5 duplicate rows
Missing: age, salary, phone
Contradictory: age vs birth_y
Email whitespace + mixed ca:

FUZZY MATCHING — FOR NEAR-DUPLICATES

```
# Near-duplicate city names: 'Bangalore' vs 'Bengaluru'
from fuzzywuzzy import fuzz process
cities = df['city'].unique()
# Score ≥ 80 → same city, apply canonical map
match_score = process.extractOne('Bengaluru', cities, scorer=fuzz.ratio)
# Validate with assertions – catch bugs before model does
assert df.duplicated().sum() == 0 "Duplicates still exist!"
assert df['email'] == df['email'].str.lower() all "Email not lowercase!"
```

PRODUCTION RULE SUMMARY

Duplicates → `df.duplicated().sum()` → `drop_duplicates()`

Inconsistent case → `value_counts()` → `.str.strip().str.title()`

Near-duplicates → fuzzy matching → canonical map

Wrong dtypes → `df.dtypes` → explicit cast

Contradictory records → column cross-check → trust ground truth column

Validation → assert at the end of every pipeline step

The Production Rule

Every cleaning step must be **reproducible and written as a function**, not a one-time script. New data arrives every day. Your pipeline runs on it automatically. If cleaning only lives in a notebook you ran once, it will break in production within a week.

INTERVIEW QUESTIONS

DAY 07 · Q1

AMAZON — DATA ENGINEER INTERVIEW

You receive a production dataset with 2 million rows and are asked to clean it before retraining. Walk me through your process.

ANSWER DIRECTION

Start with a full audit: shape, dtypes, null counts, duplicate count, `value_counts` on key categorical columns. Prioritize by column importance to the model. Write every cleaning step as a function — not a one-time fix — because new data arrives daily. Add assertions at the end of each step to confirm the fix worked. The sequence is what they're testing: audit → duplicates → formatting → dtypes → contradictions → validate.

DAY 07 · Q2

FLIPKART — ML ENGINEER ROUND

Your city column has "Mumbai", "mumbai", "MUMBAI", "Mumbai " — how do you handle this and why does it matter for the model?

ANSWER DIRECTION

This is an entity resolution failure. A model treats each variant as a separate category, learns false patterns, and fails to generalise. Fix with `strip()` + `title()` for basic normalisation. Use fuzzy matching (`fuzzywuzzy`) for near-duplicates like "Bangalore" vs "Bengaluru". Always build a canonical mapping and apply it consistently across train and production data. Without this, a city-level marketing model would miss the majority of the customers it was meant to target.

MNC Interview Q&A

Google

Amazon

Flipkart

Paytm

Microsoft

Swiggy

Razorpay

DAY 01 • GOOGLE

Structured + unstructured + semi-structured dataset. How do you build the pipeline?

KEY POINTS

Separate ingestion pipeline per type → schema validation at each entry → feature extraction for unstructured → join vectors before model.

DAY 02 • FLIPKART

Star rating encoded as numerical. What's wrong?

KEY POINTS

Ordinal, not numerical. For linear models: one-hot. For tree models: ordinal encoding is fine.

DAY 03 • AMAZON

Target skewness = 2.3. What do you do for linear regression?

KEY POINTS

Log/Box-Cox transform → train on transformed → inverse-transform predictions → verify skewness dropped.

DAY 04 • MICROSOFT

Walk me through your EDA process on an unseen dataset.

KEY POINTS

shape → dtypes → nulls → univariate → target balance → bivariate → pairplot → hypothesis list before model.

DAY 04 • RAZORPAY

Fraud model has 99.5% accuracy but business says useless. Why?

KEY POINTS

Accuracy paradox — predicts majority class only. 0.5% fraud → 99.5% accuracy for "no fraud" always. Use F1 / AUC-ROC instead.

DAY 05 • GOOGLE

35% missing values. Impute or drop?

KEY POINTS

Identify missingness type first → check feature importance → MCAR: mean OK, MAR: KNN, MNAR: indicator column. Fit imputer on train only.

DAY 05 • SWIGGY

Why is mean imputation dangerous for MNAR data?

KEY POINTS

MNAR means value is missing because of what it would have been. Assigning mean introduces systematic bias the model learns and amplifies. Clinical ML example: healthy patients have missing lab tests → imputing mean implies average health = low risk. False.

DAY 06 • PAYTM

8% of training data are outliers. What is your process?

KEY POINTS

Domain check first: errors → remove. Real extreme events → keep + evaluate separately. If needed: two models. Never remove based on statistics alone.

DAY 07 · AMAZON

2M row dataset. Walk me through your cleaning process.

KEY POINTS

Audit first (shape, dtypes, nulls, duplicates, value_counts). Prioritise by column importance. Write every step as a function. Assert at each step end. Sequence: audit → duplicates → formatting → dtypes → contradictions → validate.

DAY 07 · FLIPKART

City column has 6 variants of "Mumbai". How do you fix it and why does it matter?

KEY POINTS

Entity resolution failure — model treats each variant as a new category. Fix: strip() + title() for basic normalisation, fuzzy matching for near-duplicates, canonical map applied consistently across train and production data.

Self-test before Week 2:

Without notes — explain MCAR, MAR, MNAR with one example each. Name 3 outlier detection methods and when each breaks. Name the 5 data cleaning issues. If any feel shaky, go back to that day.

Week 1 in one sentence:

"Understand your data deeply enough to make informed decisions at every downstream step. The best ML engineers win here — not at model selection."

— #42DaysOfML

Week 2 starts with Feature Engineering → Feature Selection → Encoding & Scaling
Full notebooks: github.com/Vaishnavijagtap18/42-days-aiml-challenge · #42DaysOfML